

Genetic Algorithm-Based Non-Preemptive CPU Scheduling with Arrival Time Constraints in xv6

Darsh Thakur, Shaurya Sharma, Devansh Singh
Department of Computer Science
Shiv Nadar University
Noida, India

Abstract—Process scheduling sits at the heart of any operating system, and choosing the right algorithm has a direct effect on how responsive and efficient the system feels to users. Classic algorithms such as First-Come-First-Served (FCFS), Shortest-Job-First (SJF), Round Robin (RR), and Shortest Remaining Time First (SRTF) are well understood but each carries well-known trade-offs: FCFS suffers from the convoy effect; SJF and SRTF require prior knowledge of burst times; RR provides fairness at the cost of high average waiting time. This paper describes a Genetic Algorithm (GA)-based scheduler built and tested inside xv6, a RISC-V teaching OS. We extended the kernel with a new system call (`getprocinfo`) that exposes live process metadata to user space, then ran the GA against actual running processes. Our fitness function incorporates a squared deadline-penalty term alongside waiting time and turnaround time. Under this combined objective the GA outperforms all four baselines. Experimental results on 13 concurrent processes show the GA achieving a combined cost of 508.92, versus 524.45 for SJF, 542.89 for SRTF, 942.45 for FCFS, and 1413.08 for RR — an improvement of 2.9% over the best non-preemptive baseline (SJF) and 6.3% over the best preemptive baseline (SRTF) on the combined metric.

Index Terms—CPU scheduling, genetic algorithm, xv6, non-preemptive scheduling, round robin, SRTF, arrival time, deadline-aware scheduling, operating systems

I. INTRODUCTION

Every general-purpose operating system must decide, at any given moment, which of the ready processes gets to use the CPU next. This decision, repeated thousands of times per second, has a measurable impact on application responsiveness, batch throughput, and fairness. Despite decades of research, no single algorithm dominates across all workloads, and the problem of finding an optimal schedule for n processes is, in general, NP-hard [?].

FCFS is the simplest policy: processes run in the order they arrive. It is easy to implement and fair in a loose sense, but a single long-running process arriving slightly before a batch of short ones can force all the short ones to wait—an effect commonly called the convoy effect [?]. SJF avoids this by always picking the shortest available job, which is provably optimal for average waiting time when all burst times are known. Round Robin (RR) improves fairness by time-slicing the CPU among all ready processes, but this comes at the cost of higher average waiting time due to frequent context switches and queue cycling. SRTF, the preemptive variant of

SJF, goes further by preempting a running process whenever a shorter job arrives, achieving the best possible average waiting time among all preemptive algorithms. Despite this theoretical superiority, none of these algorithms reason about per-process deadlines, which means they can all produce schedules where a small number of long processes miss their deadlines by catastrophically large margins [?].

Metaheuristic approaches, and genetic algorithms (GAs) in particular, have attracted interest as a way to explore the combinatorially large space of possible schedules without exhaustive enumeration [?]. A GA maintains a population of candidate schedules and improves them over many generations through selection, crossover, and mutation—operators loosely analogous to biological evolution. The approach naturally handles multi-objective problems, making it straightforward to optimise for waiting time, turnaround time, and deadline compliance simultaneously, something that no single-criterion greedy algorithm can do.

What makes our work different from prior simulation-only studies is the choice of evaluation platform. We implemented the scheduler entirely inside xv6 [?], a minimal RISC-V operating system used widely in OS courses. By adding a kernel system call that exports real process table data, we were able to run the GA against actual live processes rather than hypothetical test cases. This exposes practical concerns—stack size limits, integer-only arithmetic, kernel locking protocols—that simulation-based studies tend to overlook.

The remainder of the paper is structured as follows. Section II defines the problem formally. Section III describes the kernel extensions we made. Section IV presents the GA design. Section V explains the multi-objective fitness function and the role of the squared deadline penalty. Section VI reports experimental results comparing GA against FCFS, SJF, RR, and SRTF, and Section VII concludes.

II. PROBLEM FORMULATION

We consider a set of n processes $\mathcal{P} = \{P_1, P_2, \dots, P_n\}$. Each process P_i is described by three parameters:

- AT_i — arrival time (the tick at which P_i becomes ready)
- BT_i — burst time (total CPU ticks required)
- D_i — soft deadline (the tick by which P_i should finish)

The scheduler is non-preemptive: once a process begins executing it runs to completion without interruption. At any moment only processes that have already arrived may be selected. These two constraints together mean the schedule is not simply a permutation of all processes; it is a permutation that must be simulated forward in time to be evaluated correctly.

For a given execution order σ , let S_i be the time at which P_i starts and $F_i = S_i + BT_i$ be the time at which it finishes. Standard metrics are defined as:

$$WT_i = S_i - AT_i \quad (1)$$

$$TAT_i = F_i - AT_i \quad (2)$$

$$L_i = \max(0, F_i - D_i) \quad (3)$$

where L_i is the *lateness* of process P_i . Our objective is to find a schedule σ^* that minimises a weighted combination of these three metrics:

$$\text{Cost}(\sigma) = w_1 \cdot \overline{WT} + w_2 \cdot \overline{TAT} + w_3 \cdot \overline{L^2} \quad (4)$$

where $\overline{WT}$ and $\overline{TAT}$ are the averages over all processes, $\overline{L^2}$ is the average squared lateness, and $w_1 = 0.4$, $w_2 = 0.2$, $w_3 = 0.4$. The choice to *square* the lateness term is deliberate and is discussed in Section V.

III. KERNEL EXTENSIONS TO xv6

Running the GA against real processes required exposing information from the kernel process table to user space. xv6 is deliberately minimal—it has no `/proc` filesystem and no mechanism for querying process state from user programs—so we added the necessary infrastructure ourselves.

A. New Fields in `struct proc`

We added two integer fields to the kernel’s process descriptor:

- `ctime` — the value of the global `ticks` counter at the moment the process was created, recorded in `allocproc()` under the process lock.
- `rtime` — the total number of timer interrupts during which the process was in the `RUNNING` state, incremented once per clock interrupt in `clockintr()` as shown in Fig. 1.

B. The `getprocinfo` System Call

We registered a new system call (number 22) that copies a snapshot of the process table into user space:

```
int getprocinfo(struct procinfo *buf, int max)
```

The kernel-side handler iterates over `proc[NPROC]`, acquires each process’s spinlock before reading any field, skips `UNUSED` slots, and copies the result into a kernel-side buffer before calling `copyout()` to transfer it safely to user space. The lock is released before the `copyout()` call so that it is never held across a potentially blocking operation.

The `procinfo` structure shared between kernel and user space is defined in a new header `kernel/procinfo.h`:

```
void clockintr(void) {
    acquire(&tickslock);
    ticks++;
    wakeup(&ticks);
    release(&tickslock);

    struct proc *p = myproc();
    if (p && p->state == RUNNING)
        p->rtime++;
}
```

Fig. 1. Addition to `clockintr()` in `kernel/trap.c`. The guard `p->state == RUNNING` is essential; omitting it would increment `rtime` for sleeping or runnable processes.

```
struct procinfo {
    int pid;
    int ctime;
    int rtime;
    int state;
    char name[16];
};
```

C. Summary of Kernel Changes

Table I lists every file that was modified.

TABLE I
KERNEL FILES MODIFIED OR CREATED

File	Change
<code>kernel/procinfo.h</code>	New: shared data structure
<code>kernel/proc.h</code>	Added <code>ctime</code> , <code>rtime</code> fields
<code>kernel/proc.c</code>	Initialise fields in <code>allocproc()</code>
<code>kernel/trap.c</code>	Increment <code>rtime</code> in <code>clockintr()</code>
<code>kernel/syscall.h</code>	Assigned syscall number 22
<code>kernel/syscall.c</code>	Registered handler
<code>kernel/sysproc.c</code>	Implemented <code>sys_getprocinfo()</code>
<code>user/usys.pl</code>	Added entry ("getprocinfo")
<code>user/user.h</code>	Added prototype
<code>Makefile</code>	Added <code>\$U/_ga_sched</code> to <code>UPROGS</code>

IV. GENETIC ALGORITHM DESIGN

A. Chromosome Representation

A chromosome is a permutation of process indices $[c_0, c_1, \dots, c_{n-1}]$ where each $c_k \in \{0, 1, \dots, n-1\}$ appears exactly once. The position of an index in the permutation encodes priority: when the simulator is deciding which ready process to run next, it scans the chromosome from left to right and picks the first index whose corresponding process has already arrived.

This encoding means the chromosome does not directly specify start times; those emerge from the simulation. Two chromosomes that differ only in the relative order of processes that never overlap in time will produce identical schedules and identical fitness values, which naturally encourages the population to concentrate on decisions that actually matter.

B. Population Initialisation

The initial population of 50 chromosomes is seeded as follows:

- **Chromosome 0:** the FCFS order (sorted by arrival time). This provides a known-reasonable starting point.
- **Chromosome 1:** the SJF order (sorted by burst time, ties broken by arrival). This seeds the population with the greedy optimal for waiting time.
- **Chromosomes 2–49:** random permutations generated using a multiplicative congruential RNG seeded from the current uptime and process ID.

Seeding with FCFS and SJF ensures that the GA starts from at least two reasonable solutions and cannot be worse than both at generation zero.

C. Selection

We use tournament selection with tournament size $k = 3$. Three chromosomes are drawn uniformly at random from the population; the one with the lowest combined cost (equation 4) is returned as a parent. This is repeated independently for each of the two parents required for crossover. Tournament selection is computationally cheap and provides a controllable selection pressure without requiring the population to be sorted.

D. Crossover

Single-point order crossover (OX1) is applied. Given parents P_1 and P_2 and a randomly chosen cut point c :

- 1) Copy $P_1[0 \dots c - 1]$ directly into the child.
- 2) Scan P_2 from left to right; append each gene that does not already appear in the child until the child is full.

This repair step guarantees that the child is always a valid permutation, avoiding the need for a separate feasibility check.

E. Mutation

Swap mutation is applied with probability $p_m = 0.10$. Two positions i and j are chosen uniformly at random from $[0, n - 1]$ and their contents are exchanged. The mutation rate of 10% was chosen to maintain population diversity without disrupting good solutions too aggressively.

F. Elitism

The best chromosome seen across all generations (`best_ever`) is always copied into position 0 of the next generation before offspring fill the remaining slots. This prevents the algorithm from accidentally discarding its best-found solution during selection.

G. Time Complexity

Let P denote population size, G the number of generations, and n the number of processes. Evaluating one chromosome requires simulating the schedule in $O(n^2)$ time (for each of the n scheduling decisions we scan the chromosome linearly). The total cost of the GA is therefore:

$$O(P \cdot G \cdot n^2) \quad (5)$$

With $P = 50$, $G = 100$, and $n \leq 15$, this amounts to at most $50 \times 100 \times 225 = 1,125,000$ operations per run, which completes comfortably in under one second on xv6 running in QEMU.

V. MULTI-OBJECTIVE FITNESS AND THE SQUARED DEADLINE PENALTY

A. Why Waiting Time Alone Is Not Enough

SJF is provably optimal for average waiting time under non-preemptive, single-CPU scheduling when burst times are known [?]. Any GA that uses waiting time as its sole fitness criterion will therefore converge to SJF's solution at best, and cannot surpass it.

To give the GA genuine optimisation headroom over SJF we introduced a second objective: deadline compliance. Each process is assigned a per-process soft deadline:

$$D_i = AT_i + BT_i + \left\lfloor \frac{BT_i}{2} \right\rfloor + 2 \quad (6)$$

The slack term $\lfloor BT_i/2 \rfloor + 2$ is intentionally *small for long jobs and relatively large for short jobs*. A short job with $BT = 3$ gets slack of 3 ticks (deadline = $AT + 6$), whereas a long job with $BT = 24$ gets slack of 14 ticks (deadline = $AT + 38$). Under SJF a long job arriving early may be delayed by dozens of shorter jobs and finish 80–100 ticks past its deadline. The GA, which optimises the combined cost, learns to schedule some long jobs earlier than SJF would, trading a modest increase in average waiting time for a large reduction in deadline penalty.

B. The Role of Squaring

The penalty for process P_i finishing at time $F_i > D_i$ is defined as $(F_i - D_i)^2$ rather than $(F_i - D_i)$. Squaring has two practical effects.

First, it creates a *strongly non-linear fitness landscape*. Under a linear penalty, delaying a job by one extra tick always costs the same amount regardless of how late it already is. Under a squared penalty, a job that is already 50 ticks late incurs 100 additional cost units per extra tick (derivative of x^2 at $x = 50$ is 100), making the GA strongly prefer solutions that prevent large lateness values even at the cost of slightly more moderate lateness elsewhere.

Second, it ensures that SJF's characteristic failure mode—a small number of long jobs missing their deadlines by very large margins—is heavily penalised relative to solutions that spread the deadline misses more evenly. Concretely, in our experiments a long job delayed 100 ticks past its deadline contributes $100^2 = 10,000$ to the penalty, whereas ten jobs each delayed 10 ticks contribute only $10 \times 10^2 = 1,000$. This asymmetry is exactly what the GA exploits.

C. Combined Fitness

The final fitness score used for selection is:

$$\text{Cost}(\sigma) = \frac{4}{10} \cdot \overline{WT}(\sigma) + \frac{2}{10} \cdot \overline{TAT}(\sigma) + \frac{4}{10} \cdot \overline{L^2}(\sigma) \quad (7)$$

All three terms are scaled by 1000 and averaged over all processes before being combined, so they are commensurable. All four baselines — FCFS, SJF, RR, and SRTF — are evaluated using the same formula so the comparison is on a strictly equal footing.

VI. EXPERIMENTAL SETUP AND RESULTS

A. Workload Generation

Because xv6’s round-robin scheduler gives every runnable process almost exactly the same number of timer ticks per unit time, a naive snapshot of running processes yields identical `rtime` deltas for all processes—an input on which FCFS, SJF, and GA all produce identical results. We addressed this in two stages.

Stage 1: Windowed sampling. Instead of reading `rtime` once, `ga_sched` calls `getprocinfo` twice, separated by a 100-tick sleep. The burst proxy for each process is the $\text{delta } \Delta r_i = \text{rtime}_2 - \text{rtime}_1$. Only processes present in both snapshots with $\Delta r_i > 0$ are included in the scheduling problem. This eliminates sleeping processes that contribute no CPU work during the window.

Stage 2: Burst augmentation. Even with windowed sampling, the deltas remain nearly uniform (6–7 ticks each) due to the fairness of xv6’s scheduler. We therefore apply a deterministic burst-multiplier pattern indexed by each process’s creation-time rank:

$$\text{multipliers} = [1, 4, 8, 2, 6, 3, 7, 2, 5, \dots]$$

with a base unit of 3 ticks. This produces burst times of 3, 12, 24, 6, 18, 9, 21, ... ticks, a realistic mix of short, medium, and long jobs. Arrival times are compressed into a window equal to 40% of total burst time, ensuring the CPU is always busy and processes genuinely compete for it.

B. GA Parameters

Table II lists the parameters used in all experiments.

TABLE II
GENETIC ALGORITHM PARAMETERS

Parameter	Value
Population size (P)	50
Generations (G)	100
Tournament size (k)	3
Mutation probability (p_m)	0.10
Sampling window	100 ticks
Fitness weights (w_1, w_2, w_3)	(0.4, 0.2, 0.4)

C. Results

Table III shows a representative run with 13 schedulable processes. Arrivals ranged from 0 to 58 ticks; burst times ranged from 3 to 24 ticks; total makespan was 160 ticks for non-preemptive algorithms (FCFS, SJF, GA) and also 160 ticks for RR and SRTF in this run.

Several observations are worth highlighting.

TABLE III
SCHEDULING PERFORMANCE COMPARISON (13 PROCESSES)

Algorithm	Avg WT	Avg TAT	Combined	Makespan
FCFS	44.77	57.00	942.45	160
RR (quantum=4)	56.23	68.46	1413.08	160
SJF	26.31	38.54	524.45	160
SRTF	24.85	37.08	542.89	160
GA	27.92	40.15	508.92	160

- **GA vs FCFS.** FCFS has the highest combined cost (942.45), nearly 85% above GA’s 508.92. The convoy effect is clearly visible: long-burst jobs arriving early stall all subsequent processes and cause widespread deadline misses that compound through the squared penalty term.
- **GA vs RR.** Round Robin performs worst overall (combined 1413.08) despite its fairness guarantee. The time-quantum of 4 ticks causes frequent context switches, inflating waiting times (56.23 ticks average) and turnaround times. Critically, the fragmented execution pattern pushes finish times for all processes far past their per-process deadlines, resulting in severe squared penalties.
- **GA vs SJF.** GA’s raw average waiting time (27.92) is slightly *worse* than SJF’s (26.31). This is expected and by design: the GA deliberately accepts a small wait-time penalty to move long-deadline jobs earlier, reducing squared deadline penalties significantly. The combined cost drops from 524.45 (SJF) to 508.92 (GA), a 2.9% improvement.
- **GA vs SRTF.** SRTF achieves the lowest raw waiting time (24.85) of all five algorithms, as expected for the theoretically optimal preemptive scheduler. However, its combined cost (542.89) is *higher* than both SJF and GA. This is because SRTF’s aggressive preemption continuously delays long jobs whenever any shorter job arrives, causing those long jobs to miss their tight deadlines by large margins. The squared penalty on those large misses outweighs SRTF’s waiting-time advantage. GA beats SRTF on combined score by 6.3%.
- **Convergence.** The GA converges within 25 generations. The initial population is seeded with the FCFS and SJF orderings, so generation 0 already starts from a near-optimal solution. Subsequent generations refine the ordering to protect tight deadlines on long-burst processes.

D. GA Recommended Execution Order

The GA’s recommended order for the 13-process run was:

$$\begin{aligned} P4(3) \rightarrow P6(12) \rightarrow P10(6) \rightarrow P12(18) \rightarrow \\ P14(9) \rightarrow P24(3) \rightarrow P20(6) \rightarrow P22(15) \rightarrow \\ P32(6) \rightarrow P28(12) \rightarrow P8(24) \rightarrow P18(21) \rightarrow \\ P30(24) \end{aligned}$$

(Numbers in parentheses are burst times.) The GA places the two longest jobs — P8 and P30, both with burst=24 — at positions 11 and 13 respectively, but crucially it inserts P8 *before* P18 and P30, rather than leaving them both until

last as SJF would. Similarly, P18 (burst=21) is moved ahead of P30 (burst=24). This reordering means those long jobs finish closer to their tight deadlines, avoiding the catastrophic squared penalties that SJF and SRTF both accumulate by postponing them until every shorter job has run.

VII. DISCUSSION

A. Limitations

The burst augmentation step, while necessary for the GA to operate on a non-trivial scheduling problem, does mean that the burst times used by the GA are not the actual CPU requirements of the real processes. The GA therefore produces a schedule that is optimal for the *augmented* problem rather than the real one. Addressing this would require either a kernel-level scheduler that uses the GA's output directly, or a more sophisticated burst prediction mechanism such as exponential averaging of historical rtime values.

Additionally, the RR and SRTF simulators implemented in user space are approximations. The RR simulator uses a fixed quantum of 4 ticks and a simple round-robin queue, which does not account for the overhead of context switching that a real kernel would incur. The SRTF simulator uses an event-driven model that jumps between arrival events rather than running tick by tick, which is functionally correct but does not capture the overhead of frequent preemptions in a real system. Both simulators use the same deadline and combined-cost formula as the GA, so the comparison is fair in terms of the objective function, even if the raw waiting times may differ slightly from what a kernel-level implementation would produce.

Finally, the current implementation does not handle process creation or termination during the sampling window. Processes that appear in only one snapshot are discarded; processes that terminate mid-window are excluded because their second-snapshot state is ZOMBIE and filtered out. A production implementation would need to handle these cases more carefully.

VIII. CONCLUSION

We have presented a complete, working implementation of a Genetic Algorithm-based CPU scheduler integrated with a real operating system kernel. By adding two fields to xv6's process descriptor and a new system call to export process metadata, we enabled a user-space GA to observe live kernel processes and propose optimised execution orders.

We compared the GA against four baselines: FCFS, SJF, Round Robin (quantum = 4 ticks), and SRTF. Our key finding is that a GA using a multi-objective fitness function combining average waiting time, average turnaround time, and a squared deadline-penalty term outperforms all four baselines on the combined objective. Concretely, on a 13-process workload the GA achieved a combined cost of 508.92 versus 524.45 for SJF (2.9% improvement), 542.89 for SRTF (6.3% improvement), 942.45 for FCFS (46% improvement), and 1413.08 for RR (64% improvement).

The result against SRTF is particularly noteworthy. SRTF is the theoretically optimal preemptive scheduler for average

waiting time, yet its combined cost exceeds the GA's because it systematically delays long-burst processes until every shorter job has run, causing those long processes to miss their per-process deadlines by large margins. The squared penalty on these large misses outweighs SRTF's waiting-time advantage. This demonstrates that moving from any single-criterion greedy heuristic to a multi-objective evolutionary approach yields measurably better schedules when deadline compliance matters.

Future directions include increasing population size and generation count for larger process sets, incorporating dynamic arrivals during the GA run, and experimenting with the fitness weights to reflect different application priorities.

REFERENCES

- [1] S. R. Sakhare and M. S. Ali, "Genetic Algorithm Based Adaptive Scheduling Algorithm for Real-Time Operating Systems," Available: https://www.researchgate.net/publication/274174375_Genetic_Algorithm_Based_Adaptive_Scheduling_Algorithm_for_Real_Time_Operating_Systems
- [2] "Advanced Scheduling Techniques for Real-Time Systems," Available: <https://www.sciencedirect.com/science/article/pii/S2772941925001425>
- [3] "Developing the Best Scheduling Algorithm from Existing Algorithms for RTOS," Available: <https://www.scribd.com/document/540678776/Developing-the-best-scheduling-algo-from-existing-algos-for-RTOS>
- [4] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*. Reading, MA: Addison-Wesley, 1989.